

Feed-Forward Neural Networks

General idea

A **Feed-Forward Neural Network** is an **example of MLP**

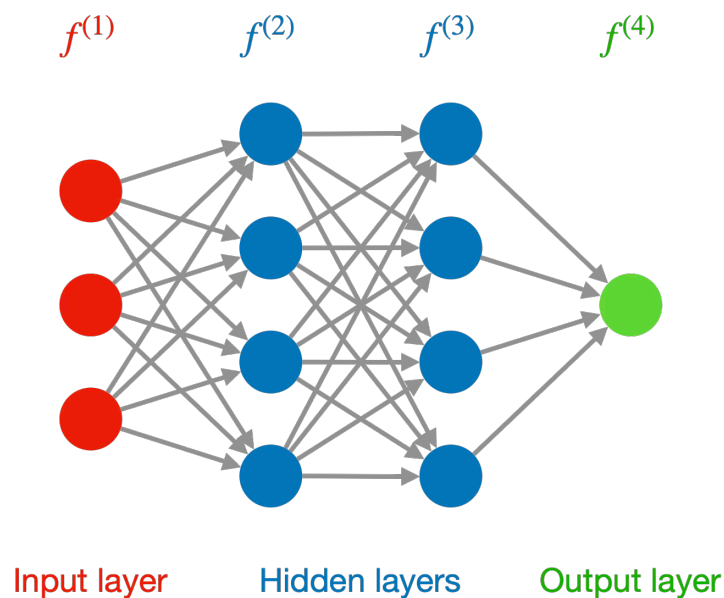
The information flows from input to output without loops

- **Represented as a DAG** (= Directed Acyclic Graph)

It **models a function** f which **results from the combination of functions** $f^{(1)}, f^{(2)}, \dots, f^{(i)}$

- **Each layer implements a function**
 - **Input layer:** receives data
 - **Hidden layers:** compute intermediate representations of data
 - **Output layer:** returns the final outcome

$$f(x) = f^{(i)}(f^{(i-1)}(\dots f^{(1)}(x)))$$



The **goal** is to **approximate some unknown ideal function** $f^* : X \rightarrow Y$, by:

- Defining a parametric mapping $f(x_i; \Theta)$ (where x_i is a datapoint, Θ is a set of parameters)
- **Learning parameters to get a good approximation of f^***

Training phase

The objective

The **aim** of training is to **optimize** Θ to **drive** $f(x; \Theta)$ **closer** to $f^*(x)$

- We **don't know** the **ground truth value for intermediate layers** → **hidden layers**
- We **only know** the **ground truth value** for the **output layer**

Difficulties and problems



Training a neural network is **similar to training** any other ML model **using gradient descent**

However:

- The **problem** becomes **non-convex**
- **No convergence** is **guaranteed**

Cost functions

Error measurement

The **error function** $\mathcal{L}$ **measures** the **distance of the network from ground truth data**

- Computes the **distance between** the **predicted value** $f_{\theta}(x)$ and the **ground truth value** y

$$L = \text{distance}(f_{\theta}(x), y)$$

- **Usual loss functions** applied in Feed-Forward Neural Networks are:

- Regression: **square loss function**

$$L_i = \frac{1}{2}(y'_i - y_i)^2$$

- Classification: **cross-entropy function**

$$L_i = - \sum_k y_k \log S(l_k) = - \log(S(l))$$

where $S(l)$ is the softmax calculation

Output layer

The 2 **main forms of output** of Feed-Forward Neural networks are:

- **Regression**: linear output units
- **Classification**: softmax

Linear output units

Usually applied in regression problems

Given a set of features h from previous layers, a layer of linear output units computes:

$$\hat{y} = W^T h + b$$

Where:

- $\hat{y}$ is the resulting prediction
- W is the weights vector and b the bias

“In regression problems, the last layer (the output layer) unit computes the weighted sum resulting from the previous layers and adds the bias”

Softmax

Usually applied in the **classification problems**

The **softmax function computes normalized probabilities** from **logits**

(= **not normalized logarithmic probabilities**) as follows:

$$S(l_i) = \frac{e^{l_i}}{\sum_k e^{l_k}}$$

Where:

- l_i is the logit resulting from the prediction on example x_i
- e is the exponential function

“In multiclass classification, the Softmax function transforms raw scores (logits) from the final hidden layers into a probability distribution that sums up to 1.”

⚠ The loss function optimizes the log of the soft max in order to:

- Preserve stability: the logarithm log zeros the natural exponents e
- Avoid the vanishing gradient: the derivative is linear and doesn't saturate

Activation functions in hidden units

Every **hidden unit** computes 3 **steps**:

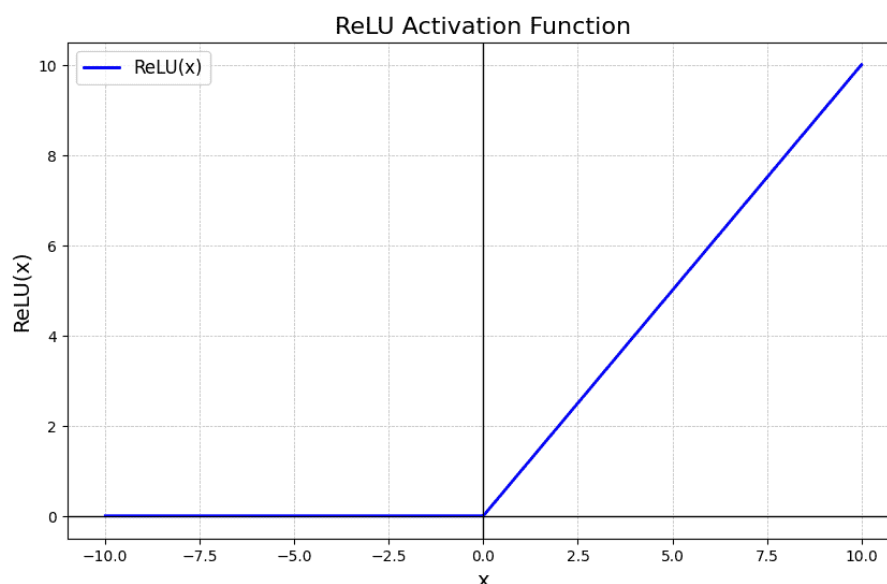
- **Accepts** an **input** x (from the previous layer)
- **Computes** an **affine transformation** $z = W^T x + b$
- **Applies** a **non-linear activation function** $h(z)$ to get the unit's output

The activation function can have multiple forms, such as: **ReLU**, **Sigmoid**, **Tanh**

ReLU (Rectified Linear Units)

The **ReLU** function is defined as follows:

$$h(x) = \max(0, x) \quad \text{which equals to} \quad h(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases}$$



We observe that:

- Computes a **constant gradient**: 1 if active, 0 if not
- Is **easy to compute and to optimize**, as **doesn't saturate for positive values**
 - Doesn't suffer from the vanishing gradient for positive values

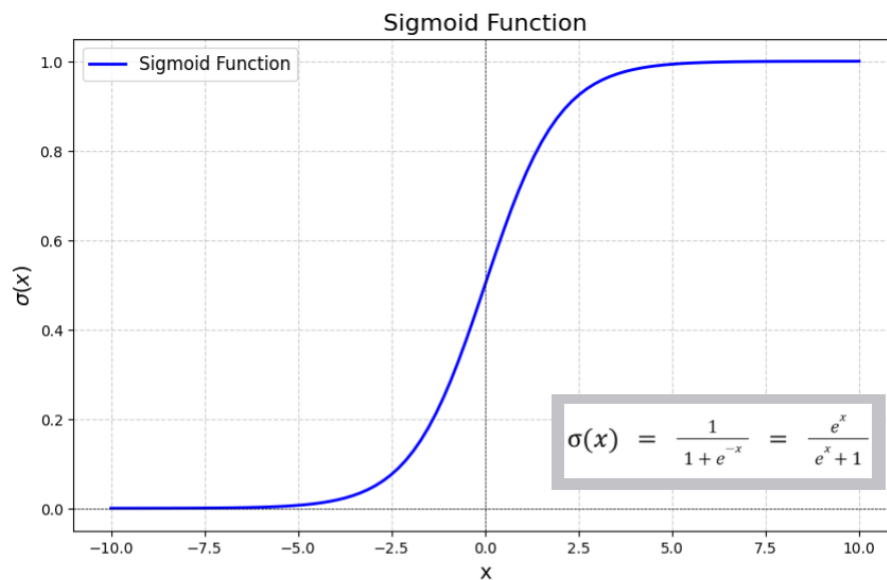
Many variants exist, to avoid units to suffer from the vanishing gradient and block at gradient 0:

- Leaky ReLU and PreLU maintain a small gradient for negative values
- Randomized Leaky ReLU maintains a random small gradient for negative values

Sigmoid

The **sigmoid** function is defined as follows:

$$h(x) = \frac{1}{1 + e^{-x}}$$



We observe that:

- **Computes values between 0 and 1** (included)

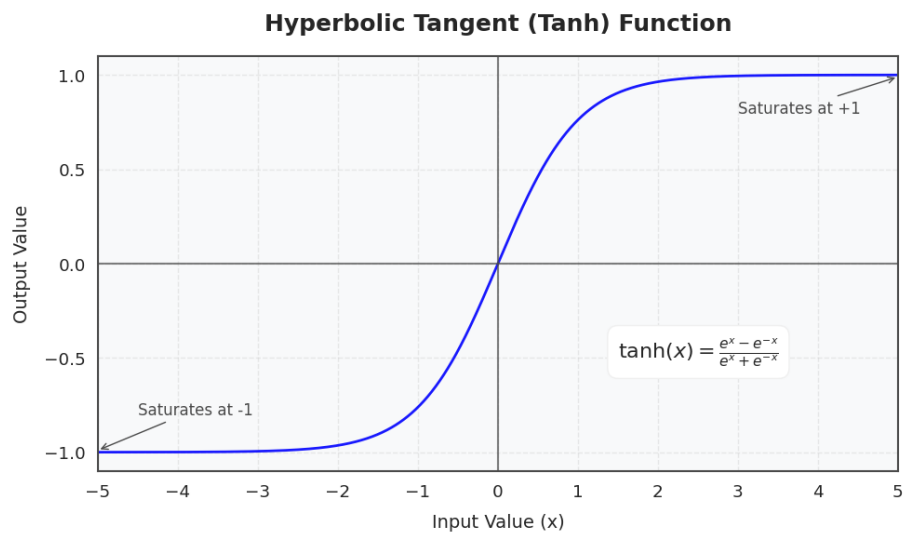
However:

- **Saturates across the majority of the domain** → risk of the vanishing gradient
- The **gradient is sensitive only** when the **input is closer to 0**

Tanh

The **sigmoid** function is defined as follows:

$$h(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



We observe that:

- **Computes values between -1 and 1 (included)**

However:

- **Saturates across the majority of the domain** → risk of the vanishing gradient
- The **gradient is sensitive only** when the **input is closer to 0**

Backpropagation

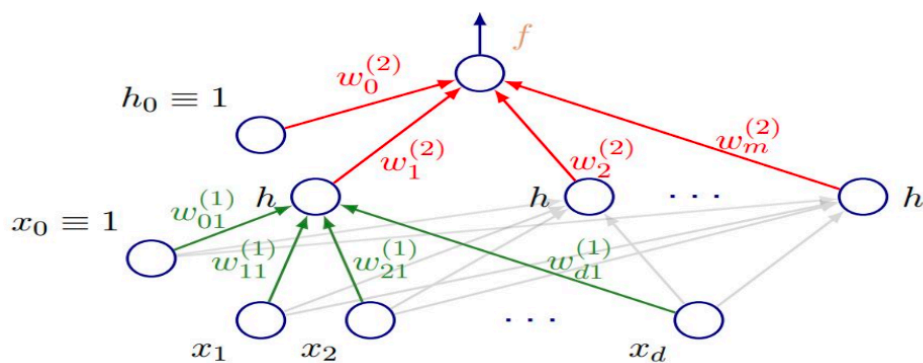
General idea

3 main steps occur:

- **Feed-Forward** propagation of the input
- **Comparison** of the **output** with the **label** and **error computation**
- **Backpropagation** of the error to **update the weights** in the hidden layers

Feed-Forward propagation

The **input** x is **propagated** and the **output** $\hat{y}$ is **computed** as a **composition** of **functions**:



$$\hat{y}(\mathbf{x}; \mathbf{w}) = f \left(\sum_{j=1}^m w_j^{(2)} h \left(\sum_{i=1}^d w_{ij}^{(1)} x_i + w_{0j}^{(1)} \right) + w_0^{(2)} \right)$$

Error computation and training

Loss function

Given a training set X the error is computed as:

$$L(X, \mathbf{w}) = \sum_{i=1}^N \frac{1}{2} (y_i - \hat{y}(x_i; \mathbf{w}))^2$$

Where:

- y_i is the **ground-truth label** associated with the example x_i
- $\hat{y}(x_i, \mathbf{w})$ is the **output** associated with the **input example** x_i

Training weights

Observe that:

- There is **no closed-form solution** → we **can't isolate w** and **calculate it**

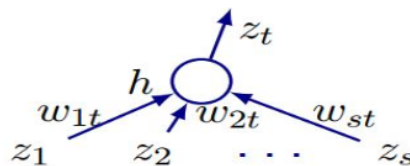
The **training** is **based on** the **gradient descent**

- We **need to evaluate the derivative** of L on single examples

Considering a linear model $\hat{y} = \sum_j w_j x_{ij}$ the derivative is $\frac{\partial L(\mathbf{x}_i)}{\partial w_j} = \underbrace{(\hat{y}_i - y_i)}_{\text{error}} x_{ij}$

General unit activation

In a **multilayer network** the **general unit activation** is computed as:



$$z_t = h\left(\sum_j w_{jt} z_j\right) \quad \text{where } h \text{ is the activation function}$$

During the Feed-Forward propagation, **the net computes the value before the activation**

$$a_t = \sum_j w_{jt} z_j$$

Backpropagation

The **loss** L **depends** on w_{jt} **only through** a_t , because:

- In the Feed-Forward propagation **the weight information is expressed only through** a_t
- When applying the back propagation, **we must differentiate based on** a_t

$$\frac{\partial L}{\partial w_{jt}} = \frac{\partial L}{\partial a_t} \frac{\partial a_t}{\partial w_{jt}} = \frac{\partial L}{\partial a_t} z_j \quad \text{using the chain rule}$$

The **gradient of the loss function** in a single general unit is the **error signal of the destination neuron** multiplied by the **input value**

Usually the error signal of a single neuron $\frac{\partial L}{\partial a_t}$ is referred to as δ_t

We have observed that for a general unit:

$$\frac{\partial L}{\partial w_{jt}} = \delta_t z_j$$

However we should distinguish 2 cases:

- Unit in the output layer
- Unit in a hidden layer

Output layer

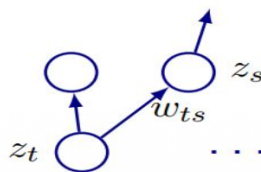
The derivative is the **difference** between the **computed output** and the **ground-truth label**

$$\delta_t = \hat{y} - y$$

Hidden layers

To compute the derivative we **need to apply backpropagation**

- From the last to the first hidden layer



$$\delta_t = \sum_{s \in S} \frac{\partial L}{\partial a_s} \frac{\partial a_s}{\partial a_t} = h'(a_t) \sum_{s \in S} w_{ts} \delta_s \quad \text{where} \quad a_s = \sum_{j: j \rightarrow s} w_{js} h(a_j)$$

Training as optimization

Optimization problem

We have observed that the **learning consists of an optimization phase**

- We **aim** to **adjust** the **weights** of the net such that the **cost function is minimized**

$$\min_{\Theta} \sum_i L(\mathbf{y}_i, f(\mathbf{x}_i; \Theta))$$

Θ is the network

Gradient Descent

💡 A **great way** to **update** the **weights** is applying **gradient descent**

- The **gradient** is the **vector of partial derivatives**

$$\nabla_w L = \left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \dots, \frac{\partial L}{\partial w_M} \right]$$

- Each partial derivative measures “how fast” the loss changes in one direction
- When the gradient is zero, all the partials are zero
 - The loss is not changing
 - Can happen in local minima and saddle points

Multiple approaches to calculate the gradient

Batch Gradient Descent - BGD

Estimates the gradient **on** the **entire training set**:

- **Stable**: is computed **using all** the available **datapoints**
- **Expensive**: **needs to compute** gradients **over the entire set** for one update

```

Input: Learning rate  $\eta_k$ , initial parameters  $\mathbf{w}$ 

while stopping criteria not met do
  Compute gradient estimate over  $N$  examples

   $\mathbf{g} \leftarrow \frac{1}{N} \nabla_{\mathbf{w}} \sum_i L(f(\mathbf{x}_i; \mathbf{w}), \mathbf{y}_i)$ 

  Apply update:  $\mathbf{w} \leftarrow \mathbf{w} - \eta_k \mathbf{g}$ 

end while
```

The learning rate changes at each step, typically **is decayed linearly**

Stochastic Gradient Descent - SGD

Estimates the gradient **on one example datapoint** per time

- **Not expensive**: the **update** occurs **after** accessing only **one** training **example**
- **Not stable**: the estimates **can be** very **noisy**

```

Input: Learning rate  $\eta_k$  , initial parameters  $\mathbf{w}$ 

while stopping criteria not met do
  Sample one datapoint from training set  $(\mathbf{x}_i, y_i)$ 
  Compute gradient estimate

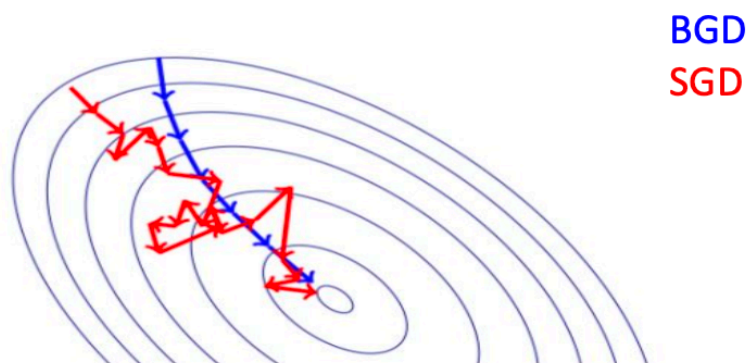
   $\mathbf{g} \leftarrow \nabla_{\mathbf{w}} L(f(\mathbf{x}_i; \mathbf{w}), y_i)$ 

  Apply update:  $\mathbf{w} \leftarrow \mathbf{w} - \eta_k \mathbf{g}$ 

end while

```

The learning rate changes at each step, typically **is decayed linearly**



BGD versus SGD visual comparison

Mini-Batch Gradient Descent

The **compromise** between batch and stochastic gradient descent

- The **gradient** is **estimated** on a **small set of** training **datapoints**
 - Is **not as expensive** as batch gradient descent
 - Is **not as noisy** as stochastic gradient descent

With parallelization of GPUs mini-batch gradient descent can be performed on different sets at the same time

Momentum

In particular situations **SGD**:

- Has **slow progress** along **flat directions**
- **Jitters** along **steep directions**

```
Input: Learning rate  $\eta_k$ , initial parameters  $\mathbf{w}$ , initial  
velocity  $\mathbf{v}$ , momentum parameter  $\alpha$   
  
    while stopping criteria not met do  
        Sample one datapoint from training set  $(\mathbf{x}_i, \mathbf{y}_i)$   
        Compute gradient estimate  
         $\mathbf{g} \leftarrow \nabla_{\mathbf{w}} L(f(\mathbf{x}_i; \mathbf{w}), \mathbf{y}_i)$   
        Compute velocity update  
         $\mathbf{v} \leftarrow \alpha \mathbf{v} - \eta_k \mathbf{g}$   
        Apply update:  $\mathbf{w} \leftarrow \mathbf{w} + \mathbf{v}$   
    end while
```

To solve **velocity $\mathbf{v}$** is introduced

- Has an **exponential decay**
- Is the **average** of the **negative gradient**

Adaptive Learning Rate

Features can **vary** in **importance and frequency**

- Applying the **same learning rate to all of them** isn't ideal
- → We **should adapt** it to each feature
 - The learning rate is the **most difficult hyperparameter to set**